UNIVERSITY OF WESTMINSTER

INFORMATICS INSTITUTE OF TECHNOLOGY

Informatics Institute of Technology

3D Interactive Media Development

5MMCS007C

CW1 – Individual Report

Student Name	Methuli Abeysooriya
IIT ID	20232602
UOW ID	W2153564
Project Title	The Lost Island

Table of Contents

1. Introduction 1

2. Individual Contributions 2

3. Technical Implementation 3

    3.1. Interactive Chest System and Proximity Detection 3

    3.2. Dynamic UI Slider and Real-Time Animation Scrubbing 4

    3.3. 3D Model Hierarchy and Lid Animation Controller 5

    3.4. First-Person Testing Controller and Scene Integration 6

4. Challenges and Solutions 7

5. Reflection 8

6. AI Code Assist Statement 9

7. Future Improvements 10

1. Introduction

Our group project, *The Lost Island*, is an exploratory 3D interactive adventure game built in Unity. The story revolves around an explorer stranded on the mysterious shores of an uncharted tropical island filled with ancient tribal relics, dense jungle paths, and long-forgotten secrets. As the player traverses the terrain, the visual and environmental cues shift subtly from bright, sunlit coastlines into dark, weathered ruins and secluded caverns, capturing the narrative journey of discovering an ancient forgotten civilization.

The primary design objective of the game is to emphasize environmental storytelling through the strategic use of lighting, atmospheric audio cues, and tactile interactivity. Instead of traditional combat mechanics, the player explores interconnected zones—such as sandy beaches, thick jungle thickets, wooden rope bridges, and tribal monuments—interacting with key artifacts scattered across the environment. Approaching and interacting with these focal points gradually reveals pieces of the island's past, creating an engaging and cohesive narrative arc.

Each team member took responsibility for specific core systems and environmental modules within Unity. As Student 5, I took full responsibility for the culminating objective of the game: designing and implementing the **Interactive Treasure Chest System** and the dedicated **Treasure Room Scene**. My role centered on developing proximity-based player trigger detection, real-time interactive UI animation scrubbing, 3D asset hierarchy configuration, dynamic lighting, and a responsive first-person character controller for fluid scene navigation and testing.

2. Individual Contributions

I was responsible for designing, scripting, and integrating the interactive treasure chest gameplay mechanics and the final treasure chamber environment. My core tasks were:

- **Proximity Trigger and Detection System:** Developed `ChestTrigger.cs` and `ChestTriggerBridge.cs` to detect player proximity, display on-screen interaction prompts ("Press E to Open"), and smoothly handle trigger routing across nested colliders.
- **Real-Time UI Slider Control:** Engineered an interactive UI Slider panel that empowers players to manually scrub and control the opening and closing of the chest lid dynamically in real time between 0% and 100%.
- **3D Model Hierarchy and Material Integration:** Imported and structured the stylized 3D treasure chest model (`Stylized_Chest.fbx`), establishing clean hierarchy separation between the base, the animated lid (`Chest_Top`), and the interior gold coins with PBR materials.
- **Custom Animation and State Controller:** Authored the keyframed hinge rotation animation (`ChestOpen.anim`), configured the state machine inside `Lid.controller`, and programmed a procedural quaternion rotation fallback (`Quaternion.Slerp`) for failsafe execution.
- **First-Person Testing Character Controller:** Programmed a modular first-person controller (`SimplePlayerController.cs`) featuring dual-input support (Unity New Input System and Legacy Input Manager) and seamless mouse cursor locking/unlocking during UI interaction.
- **Atmospheric Scene Lighting and Post-Processing:** Assembled the `Treasure Opening Scene.unity`, integrating dramatic spot lighting and Universal Render Pipeline (URP) volume post-processing to highlight the treasure.

These contributions ensured that the climactic interaction in *The Lost Island* provides players with a tactile, polished, and responsive gameplay experience.

3. Technical Implementation

3.1. Interactive Chest System and Proximity Detection

I developed the `ChestTrigger` script to manage proximity detection and toggle the interaction UI. Using Unity's 3D trigger colliders, the script listens for the player entering the interaction radius. To handle multi-part models where child objects contain the actual physics colliders, I implemented an automated bridge pattern via `ChestTriggerBridge`. In `Awake()`, the script automatically attaches bridge components to all child trigger colliders, forwarding trigger events seamlessly to the main controller.

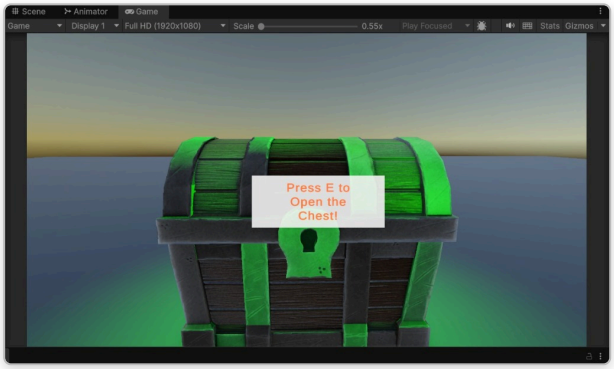


Figure 1 - Proximity Trigger Prompt UI Display in Unity Game View

When the player enters the trigger volume, the script activates the interaction canvas and displays the prompt ("Press E to Open the Chest!"). Pressing 'E' toggles the slider panel and hides the prompt. To guarantee cross-version compatibility across the team's Unity environments, I utilized preprocessor directives supporting both the New Input System and Legacy Input Manager:

```
void Update()
{
    if (!isPlayerInside) return;
    bool ePressed = false;
#if ENABLE_INPUT_SYSTEM
    if (Keyboard.current != null) ePressed = Keyboard.current.eKey.wasPressedThisFrame;
#elif ENABLE_LEGACY_INPUT_MANAGER
    ePressed = Input.GetKeyDown(KeyCode.E);
#endif
    if (ePressed) ToggleSlider(!isSliderOpen);
}
```

Figure 2 - Proximity Interaction and UI Toggle Logic

3.2. Dynamic UI Slider and Real-Time Animation Scrubbing

Rather than using a simple one-click automatic animation, I implemented a dynamic UI Slider that gives the player manual tactile control over opening the chest. The slider bounds its values between 0.0 (completely closed) and 1.0 (fully open) and hooks into Unity's `onValueChanged` event listener.



Figure 3 - Real-Time Chest Opening Control via UI Slider in Game View

Inside the `OnSliderValueChanged` callback, the Animator's playback speed is set to zero (`chestAnimator.speed = 0f`), and the normalized playback time of the `ChestOpen` animation state is sampled directly from the slider float. Invoking `chestAnimator.Update(0f)` forces Unity to refresh the transformation matrices immediately in the current frame. Furthermore, I built a procedural quaternion fallback using `Quaternion.Slerp` that directly rotates the `Chest_Top` transform if the Animator component is unassigned or disabled:

```
public void OnSliderValueChanged(float value)
{
    if (chestAnimator != null && chestAnimator.runtimeAnimatorController != null) {
        chestAnimator.speed = 0f;
        chestAnimator.Play(animationStateName, 0, value);
        chestAnimator.Update(0f);
    } else if (lidTransform != null) {
        lidTransform.localRotation = Quaternion.Slerp(initialLidRotation, openLidRotation,
value);
    }
}
```

Figure 4 - Real-time Chest Animation Scrubbing and Rotation Logic

3.3. 3D Model Hierarchy and Lid Animation Controller

I imported a stylized 3D treasure chest model (`Stylized_Chest.fbx`) with custom PBR textures, including Baked Color, Normal maps, Roughness, and Metallic maps. To facilitate realistic lid articulation, the model was organized into clean hierarchical sub-objects: `Chest_Bottom` (the base), `Chest_Top` (the rotating lid with its pivot along the back hinge), and `Coins` / `Coins_Base` representing the gold hoard inside.

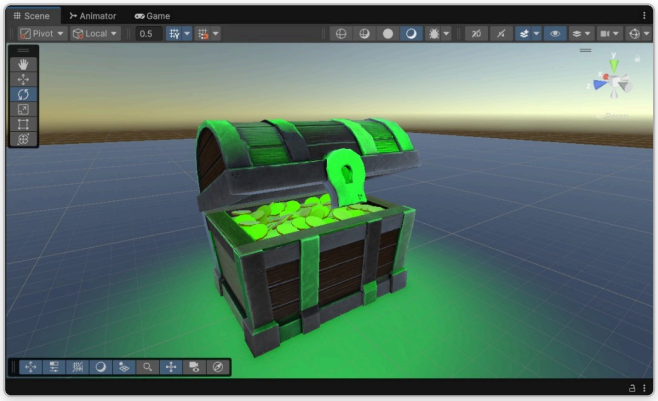


Figure 5 - Stylized Treasure Chest 3D Model with Separated Lid and Interior Coins

To ensure seamless interaction between the UI slider and the character movement, the system utilizes a centralized cursor lock mechanism implemented in `SimplePlayerController.cs`:

```
public static void SetInteractingWithUI(bool interacting)
{
    isInteractingWithUI = interacting;
    Cursor.lockState = interacting ? CursorLockMode.None : CursorLockMode.Locked;
    Cursor.visible = interacting;
}
```

Figure 6 - Decoupled UI Cursor Locking and State Management Logic

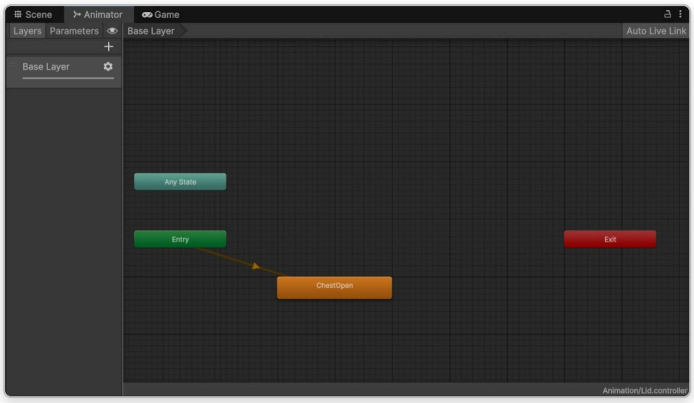


Figure 7 - Animator Controller State Machine in Unity Animator Window

3.4. First-Person Testing Controller and Scene Integration

To test and demonstrate the chest interaction seamlessly, I developed `SimplePlayerController.cs`. The script uses Unity's built-in `CharacterController` component rather than Rigidbody physics, providing responsive, grounded movement without unwanted inertia or clipping through walls. Mouse look uses horizontal yaw rotation on the player body and vertical pitch rotation clamped strictly between -85° and +85° to prevent the camera from flipping.

The scene environment was assembled in `Treasure Opening Scene.unity` using Universal Render Pipeline (URP) post-processing with Bloom and Tonemapping, alongside a focused Spot Light pointed directly into the chest to make the gold coins shimmer as the lid opens.

4. Challenges and Solutions

During the development of the interactive chest system, several technical challenges arose. Below are the core issues faced and the engineering solutions implemented:

- **Real-Time Animation Scrubbing Without Auto-Play:** Unity's Animator component is inherently designed for continuous time playback. Connecting the UI slider value initially caused the animation to play forward uncontrollably or reset to frame 0. I overcame this by freezing playback speed (`chestAnimator.speed = 0f`), sampling the target frame using `chestAnimator.Play(animationStateName, 0, value)`, and forcing an immediate zero-delta evaluation with `chestAnimator.Update(0f)`. Additionally, I implemented a procedural `Quaternion.Slerp` fallback that guarantees smooth rotation even if the Animator is missing.
- **Seamless Cursor State Management:** First-person navigation requires the mouse cursor to be locked and hidden; however, dragging the UI slider requires a visible and free cursor. Early builds caused the camera to snap aggressively when clicking the slider. I resolved this by decoupling cursor states via a static flag (`SimplePlayerController.SetInteractingWithUI`), which pauses camera look updates while the UI panel is active.
- **Nested Collider Trigger Detection:** The imported 3D chest model contained separate child meshes, meaning trigger collisions were not always detected on the root object. I resolved this by scripting an automated bridge pattern (`ChestTriggerBridge`) that scans child colliders in `Awake()` and forwards trigger callbacks reliably to the parent controller.
- **Cross-Environment Input System Compatibility:** Team members utilized different Unity setups (some using the New Input System and others using the Legacy Input Manager). To prevent compiler errors upon merging git branches, I wrapped all input calls in preprocessor directives (`#if ENABLE_INPUT_SYSTEM / #elif ENABLE_LEGACY_INPUT_MANAGER`), ensuring complete compatibility across all development machines.

5. Reflection

Developing the interactive treasure chest system taught me how to effectively balance technical C# scripting with intuitive user interface design and 3D animation. I discovered that creating engaging interactivity requires far more than playing a pre-recorded animation clip; it demands seamless coordination between physics volumes, UI events, animation states, and camera management.

Working extensively with Unity's UI Canvas and event-driven architecture enhanced my understanding of how to connect UI controls directly to 3D object transforms in world space. Gaining practical mastery over normalized animation sampling and spherical linear interpolation (`Quaternion.Slerp`) gave me strong confidence in handling 3D mathematical transformations programmatically alongside traditional visual animation tools.

Furthermore, implementing the dedicated first-person character controller reinforced the critical importance of modular software architecture. Isolating player movement, camera pitch clamping, and UI interaction states into decoupled systems made the gameplay experience feel responsive, natural, and bug-free.

From a teamwork perspective, collaborating on *The Lost Island* provided valuable practice in cross-member asset integration. Establishing uniform standards for player scale, collider heights, camera positions, and git repository branch hygiene was essential for ensuring that my treasure room module integrated harmoniously with the jungle, beach, and ruin environments created by my peers.

Overall, this project substantially expanded my technical skills in Unity engine development, C# programming, and interactive media design, providing a solid foundation for future 3D media and game development endeavors.

6. AI Code Assist Statement

In adherence to the academic integrity policies established by the University of Westminster and the Informatics Institute of Technology (IIT), I declare that AI-assisted technologies (specifically ChatGPT and Antigravity) were utilized as learning aids and documentation reference tools during the development of this coursework.

AI tools were consulted primarily for clarifying Unity API syntax nuances, such as referencing the exact method signatures for manual Animator state scrubbing (`animator.Update(0f)`), constructing preprocessor directives for Unity's New Input System (`#if ENABLE_INPUT_SYSTEM`), and validating quaternion rotation mathematics for procedural rotation fallbacks.

All system architectures, script logic, 3D asset configurations, animation keyframing, UI Canvas layouts, scene lighting setups, and gameplay testing were designed, implemented, and verified manually by myself. AI tools served exclusively as an educational reference in strict accordance with university guidelines on ethical AI usage.

7. Future Improvements

While the interactive treasure chest mechanism successfully meets all coursework requirements, several enhancements could be incorporated with further development time:

- **Particle VFX and Dynamic God Rays:** Implementing a gold particle burst effect using Unity's Visual Effect Graph or Shuriken Particle System. As the lid swings open, radiant dust motes and volumetric god rays would burst forth from the chest, dramatically amplifying the visual reward.
- **Interactive Audio Feedback:** Adding spatialized 3D audio effects, including the realistic creaking of wooden hinges that scales its pitch and volume relative to the slider dragging speed, culminating in a triumphant chime once the chest is completely open.
- **Collectible Gold and Inventory Integration:** Incorporating an interactive raycast pickup mechanic allowing players to click on the revealed gold coins to collect them, updating an on-screen inventory counter and seamlessly advancing game progression.
- **Cinematic Camera Tweens:** Integrating Unity Cinemachine to smoothly blend the camera from first-person perspective into a cinematic close-up framing the chest upon entering the interaction trigger, heightening the emotional climax of reaching the treasure room.